Nº	NOME	E-mail	Telefone
248037	Bruno Bertolini Maluf	brubemal@gmail.com	(15)99799-9909
235537	Bruno Carvalho de Oliveira	brunocarvalho.ksc@gmail.com	(15)99192-4104
248245	Guilherme Januário Serafim	guilhermeserafim.facul@gmail.com	(15)99855-4581
248224	Mateus Pasquotto Lulia Bellotto	Mateusplb27@gmail.com	(15)99820-8674
248312	Rodrigo Lopes de Oliveira Cardoso	lopesrodrigo942@gmail.com	(15)98807-4815

ATIVIDADE PRÁTICA:

(v1.0.0)

SOROCABA, SP

2026

SUMÁRIO

1 CONCEITO DE VANILLA JS 2
2 VERSIONAMENTO E DEPLOY 5
3 GESTÃO DE PROJETO 7
4 PROJETO 8
REFERÊNCIAS 9

1 CONCEITO DE VANILLA JS

Para desenvolver um e-commerce de bebidas (como vinhos, cervejas artesanais ou destilados) usando apenas Vanilla JS (JavaScript puro), HTML e CSS é um grande desafio para aprender e dominar os fundamentos da web pois, o Vanilla JS refere-se ao JavaScript puro, ou seja, o uso da linguagem em sua forma mais básica e nativa, sem a utilização de bibliotecas ou frameworks adicionais como React, Vue, Angular ou jQuery.

Um dos focos para o site e-commerce de bebidas é a organização e a acessibilidade. Para um site de bebidas, a estrutura deve ser clara para facilitar a navegação para o usuário, como por exemplo: o logotipo, a barra de busca, o ícone do carrinho de compras, tudo isso deve ser feito através da organização de estrutura de código no HTML.

Bebidas alcoólicas em geral costumam apelar para o visual, uma boa aparência chama a atenção do comprador. O CSS deve transmitir a "vibe" do produto com cores de fontes e fundo similares ao próprio produto.

Para fazer um site Vanilla sem frameworks para facilitar a criação do site, o JS será responsável por "dar vida" à loja, manipulando o DOM em vez de escrever 50 produtos no HTML um banco de dados pode ser usado para armazenar os produtos.

Figura 1 – Exemplo de HTML.

```html
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Wine Store | Seleção Exclusiva</title>
    <link href="https://fonts.googleapis.com/css2?family=Montserrat:wght@300;400;700&display=swap" rel="stylesheet">
    <link href="style.css" rel="stylesheet">
</head>
<body>

<nav>
    <div class="logo">WINE CLOSET</div>
    <input type="text" class="search-bar" id="searchInput" placeholder="Pesquisar vinhos...">
    <div class="cart-icon" onclick="toggleCart()">
        🛒 <span class="cart-count" id="cartCount">0</span>
    </div>
</nav>

<div class="container">
    <div class="main-content">
        <div class="product-grid" id="productGrid">
            </div>
    </div>
</div>

<div id="cart-sidebar">
    <h2 style="margin-bottom: 20px;">Seu Carrinho</h2>
    <div id="cartItems"></div>
    <hr style="margin: 20px 0;">
    <p>Total: <strong id="cartTotal">R$ 0,00</strong></p>
    <button class="btn-add" style="margin-top: 20px;">Finalizar Compra</button>
    <button onclick="toggleCart()" style="background: none; border: none; margin-top: 10px; cursor: pointer; color: #777;
    ">Fechar</button>
</div>

<script src="script.js"></script>

</body>
```

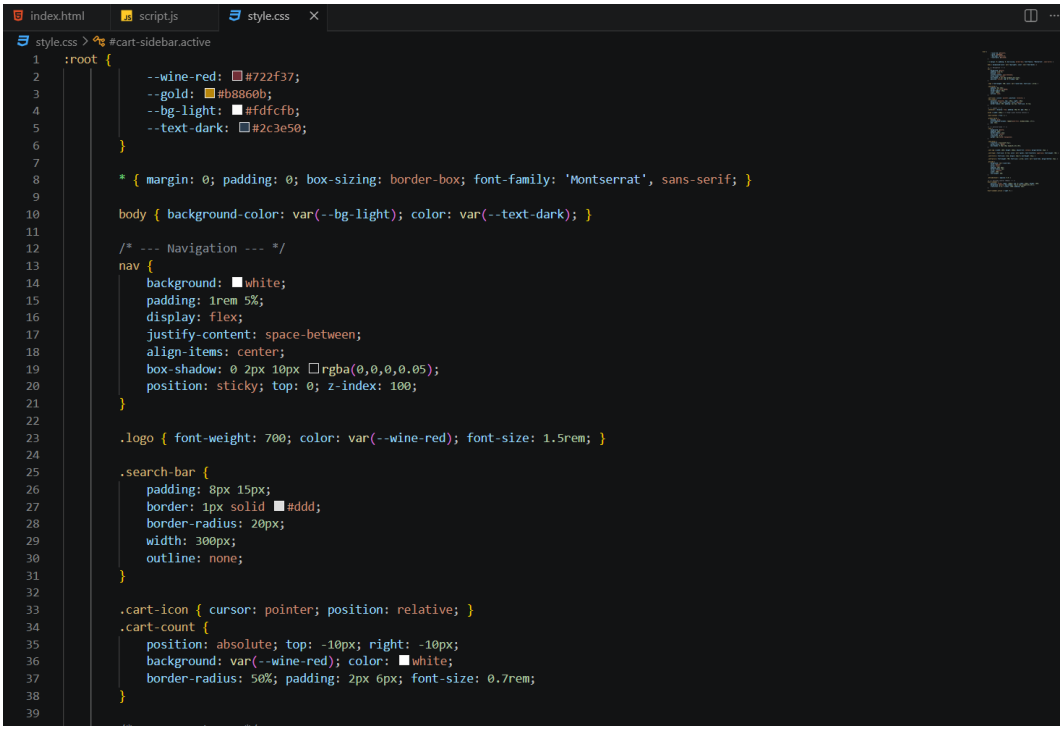
Fonte: Elaborado pelo autor (2026).

Figura 2 – Exemplo de Vanilla JS.

```javascript
// 1. Dados dos Produtos
const products = [
    { id: 1, name: "Cabernet Sauvignon Reserva", type: "Tinto", price: 89.90, img: "https://imgs.search.brave.com/
    bURJ-oSsmccyQ_wsX219KnPv6Zr_6HVbRd13bGXPpCk/rs:fit:860:0:0:0/g:ce/aHR0cHM6Ly9jZG4u/ZG9vY2Euc3RvcmUv/NjU3L3Byb2R1Y3Rz/
    LzZuZ3oyNHd3ZWd0/cG5jbmZ5Y3k0Z2V0/bDBhZnFzZWN5cXB6/M180NTB4NjAwK2Zp/bGxfZmZmZmZmLnBu/Zz92PTE3MTU0MzQ5/ODcmd2VicD0w" },
    { id: 2, name: "Chardonnay Excellence", type: "Branco", price: 75.00, img: "https://imgs.search.brave.com/
    EM-4V5ApIWhBjTblXzUFYXVa0rn0BT5XEkSpTM3wxjk/rs:fit:860:0:0:0/g:ce/aHR0cHM6Ly93d3cu/YmxhY2tleGNlbGxl/bmNld2luZWNvLmNv/
    bS9jZG4vc2hvcC9w/cm9kdWN0cy9DaGFy/ZG9ubmF5LmpwZz92/PTE2OTg4MDA4OTMm/d2lkdGg9MTk0Ng" },
    { id: 3, name: "Rosé de Provence", type: "Rosé", price: 120.00, img: "https://imgs.search.brave.com/
    RPOj6QtdGo8O5ZNt9JdNjVtRKJfGh88bAU3KX9_if20/rs:fit:860:0:0:0/g:ce/aHR0cHM6Ly9hdXJv/c2UtcHJvdmVuY2Uu/Y29tL2FwcC91cGxv/
    YWRzLzIwMjIvMDQv/QXVyb3NlLVByb3Zl/bmNlLUJhaW4tZGUt/bWlkaS1yb3NlLXNj/YWxlZC5qcGc" },
    { id: 4, name: "Malbec Argentino", type: "Tinto", price: 95.50, img: "https://imgs.search.brave.com/
    GXGwF82_3DCIKXght7wJjbWvcpFOU3lgZ_Qeh3V1JIU/rs:fit:860:0:0:0/g:ce/aHR0cHM6Ly9jZG4u/YXdzbGkuY29tLmJy/LzMwMHgzMDAvMjA4/
    Ny8yMDg3NzU2L3By/b2R1dG8vMTQyODE0/NTAwLzY1YTE4ZDdh/OGMuanBn" },
];

let cart = [];

// 2. Funções de Renderização
function renderProducts(items) {
    const grid = document.getElementById('productGrid');
    grid.innerHTML = items.map(p => `
        <div class="card">
            <img src="${p.img}" alt="${p.name}">
            <span class="card-type">${p.type}</span>
            <h3 class="card-title">${p.name}</h3>
            <p class="card-price">R$ ${p.price.toFixed(2)}</p>
            <button class="btn-add" onclick="addToCart(${p.id})">Comprar</button>
        </div>
    `).join('');
}

// 3. Lógica do Carrinho
function addToCart(id) {
    const product = products.find(p => p.id === id);
    cart.push(product);
    updateCartUI();
    toggleCart(true); // Abre o carrinho ao adicionar
}
```

Fonte: Elaborado pelo autor (2026).

Figura 3 – Exemplo de CSS.



```
index.html   script.js   style.css
style.css > #cart-sidebar.active
:root {
        --wine-red: #722f37;
        --gold: #b8860b;
        --bg-light: #fdfcfb;
        --text-dark: #2c3e50;
    }

    * { margin: 0; padding: 0; box-sizing: border-box; font-family: 'Montserrat', sans-serif; }

    body { background-color: var(--bg-light); color: var(--text-dark); }

    /* --- Navigation --- */
    nav {
        background: white;
        padding: 1rem 5%;
        display: flex;
        justify-content: space-between;
        align-items: center;
        box-shadow: 0 2px 10px rgba(0,0,0,0.05);
        position: sticky; top: 0; z-index: 100;
    }

    .logo { font-weight: 700; color: var(--wine-red); font-size: 1.5rem; }

    .search-bar {
        padding: 8px 15px;
        border: 1px solid #ddd;
        border-radius: 20px;
        width: 300px;
        outline: none;
    }

    .cart-icon { cursor: pointer; position: relative; }
    .cart-count {
        position: absolute; top: -10px; right: -10px;
        background: var(--wine-red); color: white;
        border-radius: 50%; padding: 2px 6px; font-size: 0.7rem;
    }
```

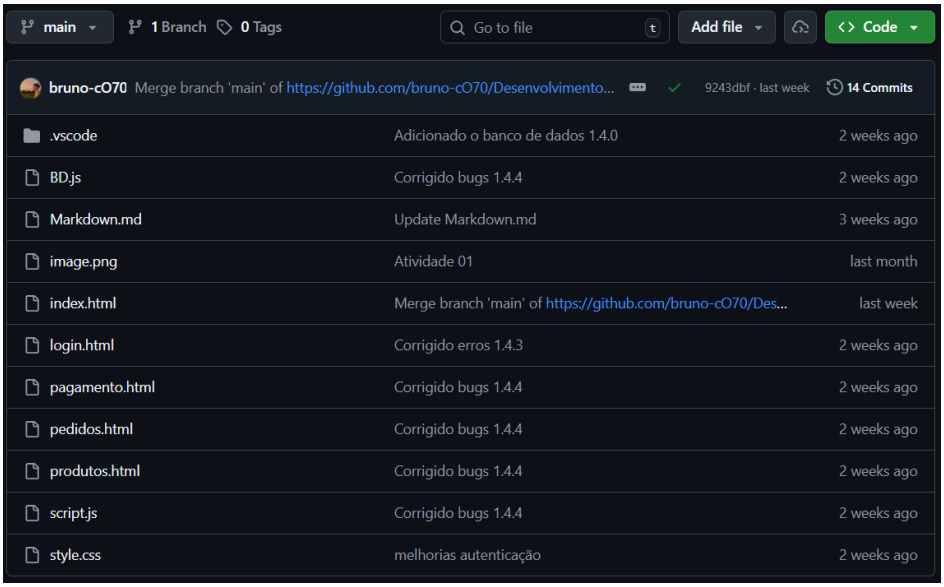
Fonte: Elaborado pelo autor (2026).

2 VERSIONAMENTO E DEPLOY

O que é Vercel ? Vercel é uma plataforma de nuvem voltada para desenvolvedores frontend. Ela é amplamente utilizada para hospedar qualquer site estático (HTML, CSS e JS puro) com extrema performance, diferente de hospedagens tradicionais onde o programador precisa subir arquivos via FTP, a Vercel funciona de forma serverless e automatizada, focando na facilidade de uso e na velocidade global de carregamento (através de CDNs).

O versionamento é o ato de registrar o histórico de alterações do seu código em plataformas como o GitHub, a diferença entre Git e GitHub é que o Git é uma ferramenta local que cria "pontos de restauração" (commits) no seu código e GitHub é uma plataforma onde você armazena esses registros na nuvem. O versionamento é crucial porque permite que você teste novas funcionalidades (em *branches* separadas) sem quebrar a versão principal que já está funcionando.

Imagem 4 – Deploy no GitHub.



Fonte: Elaborado pelo autor (2026).

O grande diferencial da Vercel é a integração com o GitHub. Esse processo é conhecido como CI/CD (Integração Contínua e Entrega Contínua).

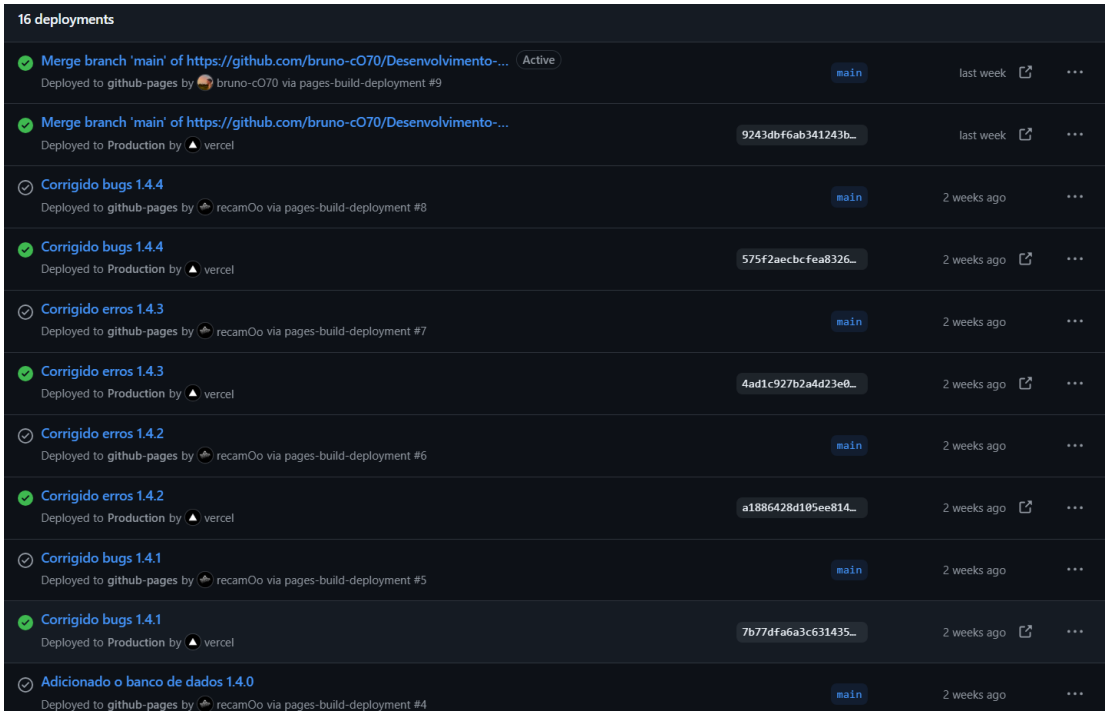
O fluxo funciona assim:

1. Código Local: Você termina uma alteração no seu HTML/CSS/JS.
2. Commit e Push: Você envia as mudanças para o seu repositório no GitHub.
3. Trigger: A Vercel detecta automaticamente que o repositório foi atualizado.
4. Build & Deploy: A plataforma baixa o novo código, otimiza as imagens e scripts, e atualiza o site em questão de segundos.

Como aplicar na prática:

- Previews de Pull Request: Se criar uma versão para testar um layout, a Vercel gera um link temporário único para você visualizar essa alteração antes de publicá-la oficialmente.
- HTTPS Automático: O certificado SSL (o cadeado de segurança) é configurado sozinho.
- Rollback: Se você subir um erro, pode voltar para a versão anterior com apenas um clique no painel da Vercel.

Figura 5 – Versionamento do GitHub.

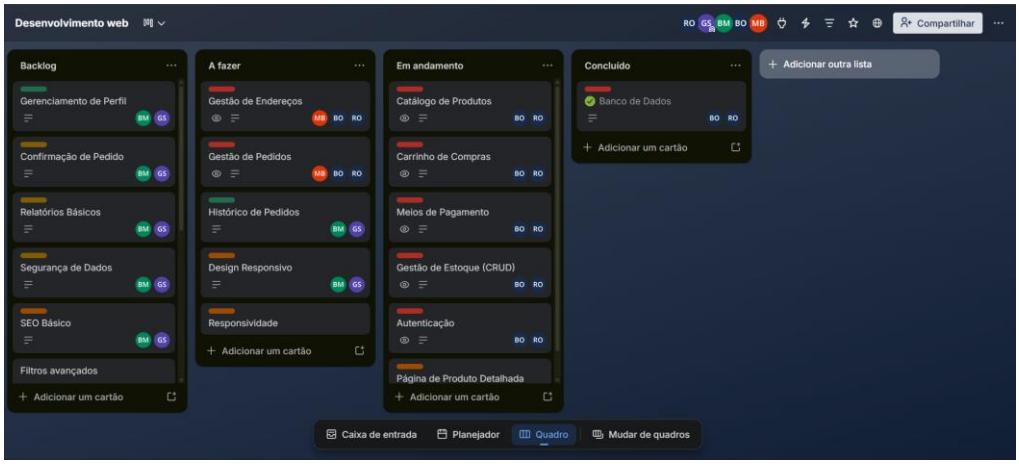


Fonte: Elaborado pelo autor (2026).

3 GESTÃO DE PROJETO

O uso do Trello, como mostrado na imagem 6, é fundamentado na metodologia Kanban, o Trello funciona como um mapa visual para que o grupo não se perca no mar de tarefas que um e-commerce exige.

Figura 6 – Kanban do projeto no Trello.



Fonte: Elaborado pelo autor (2026).

O quadro está dividido pelo ciclo de vida de uma tarefa:

- Backlog: Aqui ficam as ideias e requisitos que ainda não foram detalhados ou que não são prioridade imediata.
- A fazer: São as tarefas que o grupo decidiu que serão as próximas a entrar em desenvolvimento.
- Em andamento: Esta é a coluna mais crítica, ela mostra o que está sendo codado agora.
- Concluído: Onde mostra quais partes do site estão completas e que não precisam mais ser feitos.

4 PROJETO

- Projeto
  - GitHub: https://github.com/bruno-cO70/Desenvolvimento-Web-CP500TIN2
  - Vercel: https://desenvolvimento-web-wine-nine.vercel.app/

REFERÊNCIAS

• GIT. Documentação oficial. Disponível em: https://git-scm.com/. Acesso em: 2026.

• GITHUB. Plataforma de hospedagem de código. Disponível em: https://github.com/. Acesso em: 2026.

• VERCEL. Plataforma de deploy de aplicações. Disponível em: https://vercel.com/. Acesso em: 2026.